

birdclef-2026

## 8th Place Solution

|                 |          |
|-----------------|----------|
| <b>Rank</b>     | 8        |
| <b>Team</b>     | kazumax  |
| <b>Author</b>   | kazumax  |
| <b>Topic ID</b> | 704770   |
| <b>Version</b>  | Original |

Source: <https://www.kaggle.com/competitions/birdclef-2026/discussion/704770>

Retrieved with Kaggle CLI on 2026-07-03.

Public LB: 0.94751

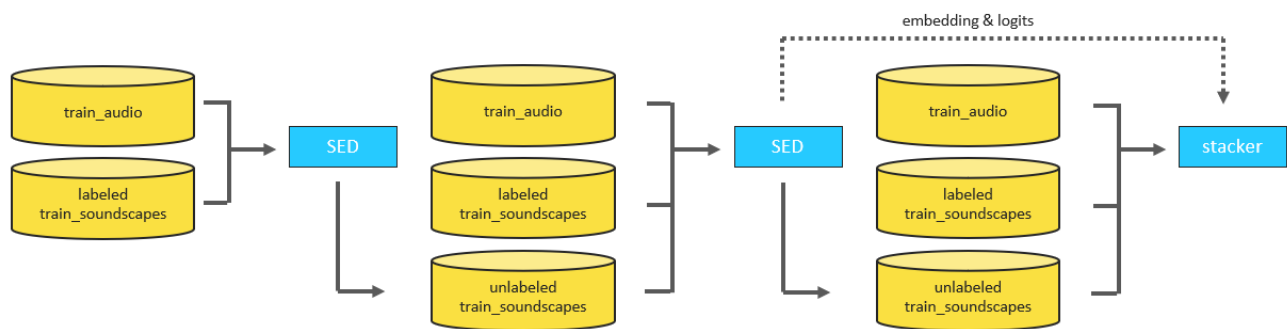
Private LB: 0.95675

First, thank you to the organizers and everyone who shared ideas during the competition.

The starting point for my solution was the public distilled SED idea shared here:

<https://www.kaggle.com/competitions/birdclef-2026/discussion/694479>

I used that idea as the starting point for 5-second Perch-distilled SED models, then added stackers to model the 60-second soundscape structure, site/hour information, temporal context, and class priors.



## TL;DR

My final submission used:

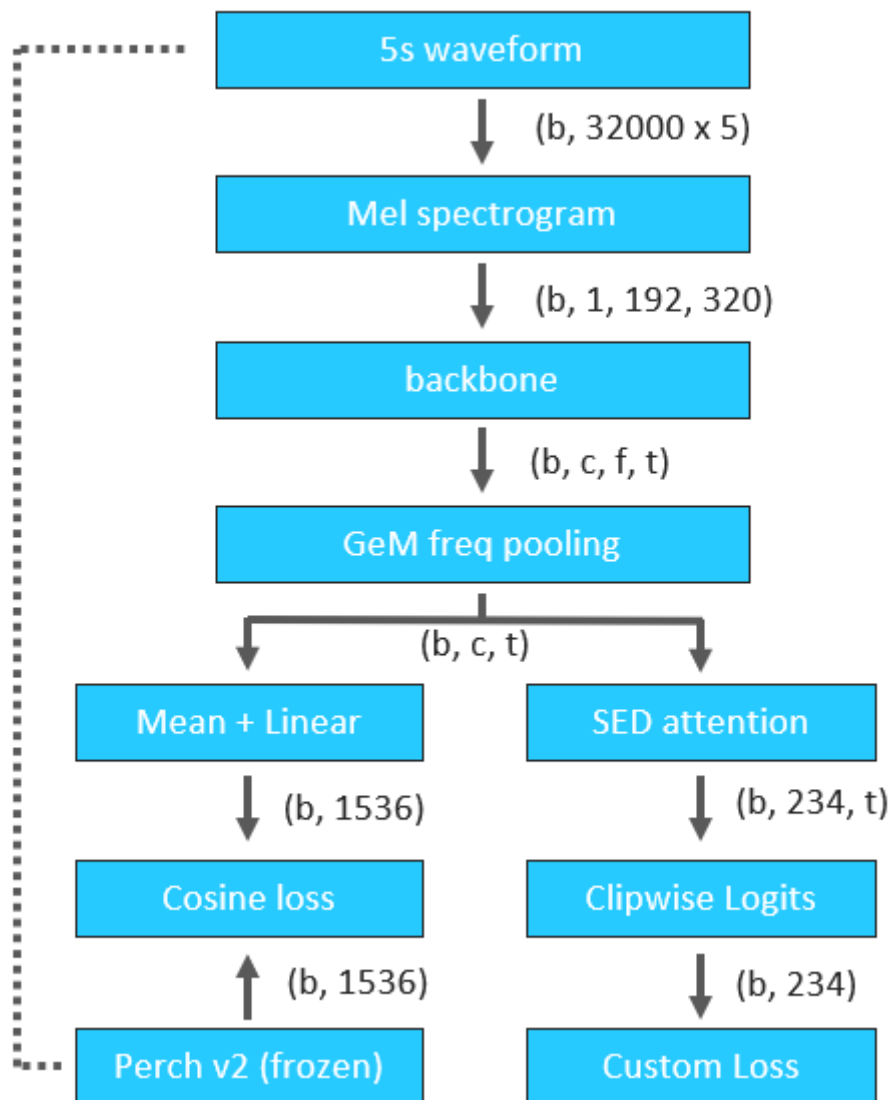
- 3-fold SED models with three timm backbones: `efficientnet_v2_b3`, `regnety_016`, and `maxxvitv2_nano_rw_256`.
- 5-second 32 kHz input audio with 192 x 320 dB-mel spectrograms.
- SED head with frequency GeM pooling.
- Online Perch v2 embedding distillation with cosine loss.
- Soft cross entropy + a weak negative-logit penalty as the supervised SED loss.
- Pseudo-labeled soundscapes for SED training.
- NN stackers with pseudo labels, and tree stackers without pseudo labels.
- Different stacker branches for sonotype and non-sonotype labels.
- OpenVINO CPU inference for SED backbones, with PyTorch heads and stackers.

## Validation

I used 3 folds. For local validation, I calculated macro AUC separately on held-out `train_audio` clips and labeled `train_soundscapes` clips, then used the mean of those two macro AUC values as the main metric.

This local metric was useful but noisy, so final model selection also used leaderboard feedback and ensemble diversity.

## Base SED Model



The base model was a 5-second SED model inspired by the public distilled SED discussion above. My final version differed in a few ways:

- Different backbones.
- Cosine embedding distillation instead of MSE.
- A custom supervised loss.
- Supervised segmentwise loss weight set to 0.0.
- Different waveform length and mel-spectrogram size.

Each 5-second clip was converted to a 192 x 320 dB-mel spectrogram and passed through a timm backbone followed by frequency GeM pooling:

backbone feature: (B, C, F, T)  
GeM frequency pooling: (B, C, T)

The SED attention head produced:

```
segmentwise logits: (B, 234, T)
clipwise logits:    (B, 234)
```

The supervised loss was applied to clipwise logits. Segmentwise outputs were still produced, but the segmentwise supervised loss weight was `0.0`.

## Creating 5-Second Training Clips

All competition audio was already 32 kHz, so each SED sample was fixed to 5 seconds, or `160000` samples.

For `train_audio`, I used random 5-second crops during training. For validation, I used the first 5-second segment.

All `train_soundscape` files were 60 seconds, so I split them into 12 non-overlapping 5-second clips.

## SED Pseudo Labels

I pseudo-labeled unlabeled soundscapes with earlier SED models. I kept a 5-second pseudo row only when its maximum class probability was at least `0.40`; within a kept row, probabilities at least `0.10` were kept as soft targets and the rest became target `0`.

I also used secondary labels from `train_audio`.

## Augmentations

I used the following augmentations:

- waveform augmentations: random shift and gain
- spectrogram transforms: `FilterAugment` and `SpecAugment`
- waveform mixing: `max_mixup_waveform` and `sumix_waveform`

## SED Loss

The final supervised SED loss was:

```
loss = Soft Cross Entropy + 0.03 * weak negative-logit penalty
```

The Soft Cross Entropy term applies `log_softmax` over the 234 classes and supports multi-hot / soft targets. Unlike BCE, it makes classes compete within a clip. I did not normalize the target vector to sum to one, so clips with multiple labels could contribute multiple positive terms.

The negative-logit penalty was a small false-positive suppressor for classes whose target was zero. It added a linear penalty when one of those logits exceeded `-5.0`, which corresponds to a sigmoid probability of about `0.0067`. Before averaging this penalty, I removed the largest 5% values so a few extreme false positives would not dominate.

After trying several loss combinations and weights, this worked best for me.

## Perch Distillation

---

The SED model was also trained to match Google Perch v2 embeddings:

```
Perch teacher:      Perch v2
target embedding:    1536 dims
distillation loss:   cosine
distillation weight: 10.0
```

This used Perch embeddings only, not Perch's class-probability outputs.

## Stacker

---

The base SED predicts each 5-second chunk independently. The stackers added 60-second sequence context, site/hour information, temporal neighbor features, and class priors.

At test time:

```
5-second rows:      L = 12
base SED logits:    (L, 234)
```

During training, `L` could vary because low-confidence pseudo rows were dropped. The NN stacker grouped rows by soundscape file and padded sequences within each batch.

I used two stacker families:

- NN stackers: GRU / LSTM sequence models.
- Tree stackers: LightGBM / XGBoost models.

I also used two output modes:

- `residual`: predict a correction to the base SED logits.
- `direct`: predict final logits directly.

In cross-validation, residual stackers worked better for non-sonotype labels, while direct NN stackers worked better for sonotype labels. The final submission used them accordingly.

## Prior Features

---

In this solution, "prior" means a label-frequency estimate from labeled `train_soundscape`s. For each class, I computed frequencies from:

- global frequency
- hour
- site
- site-hour

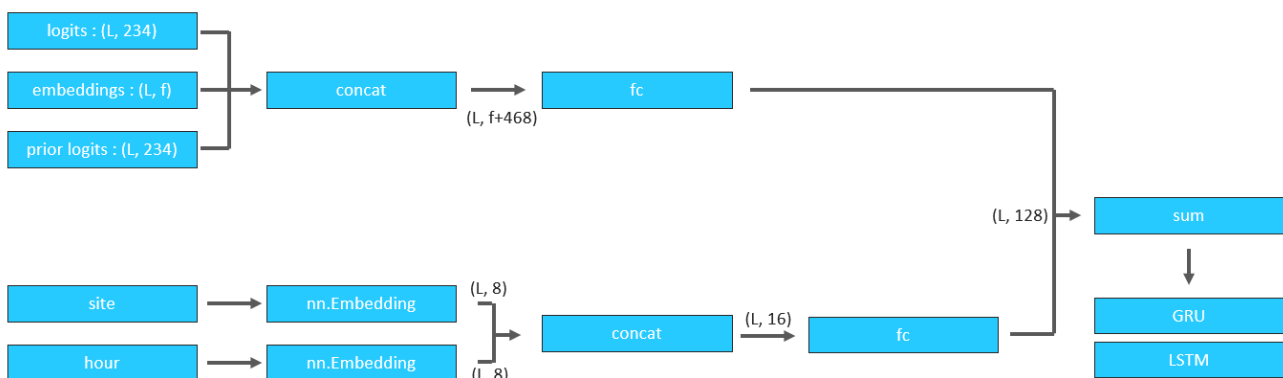
The group frequencies were smoothed toward the global frequency to avoid extreme priors from small site/hour groups.

I used two representations:

```
prior probs = probability estimates
prior logits = logit(prior probs)
```

The NN stackers used both prior probabilities and prior logits. The tree stackers used prior logits. These features gave the stackers a calibrated expectation such as "this class is common at this site/hour" or "this class is almost never observed here."

## NN Stacker



For each 5-second row, the NN stacker input was:

```
base logits      (234)
SED embedding    (f)
prior logits     (234)
prior probs      (234)
```

Here  $f$  is the pooled SED embedding dimension, which depends on the backbone.

These features were concatenated and projected to hidden size 128. Site and hour were embedded separately, concatenated, projected to hidden size 128, and added to the acoustic feature projection:

```
feature projection: (L, input_dim) -> (L, 128)
site/hour metadata: (L, 16) -> (L, 128)
sum -> bidirectional GRU or LSTM -> output head
```

The final non-sonotype NN stackers used GRU and LSTM residual models. The final sonotype NN stackers used GRU and LSTM direct models.

Unlike SED pseudo labels, the NN stackers kept only the top 5 positive pseudo classes per row:

- keep a pseudo row only if its maximum class probability is at least 0.40
- from classes with probability at least 0.10, keep only the top 5 as soft positive targets
- compute the loss for all classes in the kept row
- classes below 0.10, and classes not selected in the top 5, become target 0

## Tree Stackers

---

For LightGBM and XGBoost, training 234 separate GBMs would have been too expensive. Instead, I added `class_id` as a feature so that a single GBM could learn corrections for all classes.

The final tree stackers were residual models, and I applied them only to non-sonotype labels. They did not use SED embeddings; they relied on base SED logits and features derived from those logits.

The main tree features were:

- class id and class frequency statistics
- current base logit and derived probability
- prior logit features
- site/hour/end-second metadata
- previous and next 5-second logits for the same class
- row-level summaries from the base predictions

This training table was highly imbalanced, so I sampled negatives. I used hard negatives from high-scoring wrong classes and random negatives. The final LightGBM used up to 500000 training pairs per fold, while the final XGBoost used up to 1000000.

I did not use pseudo labels for the final LGBM/XGBoost stackers.

## Final Stacker Ensemble

---

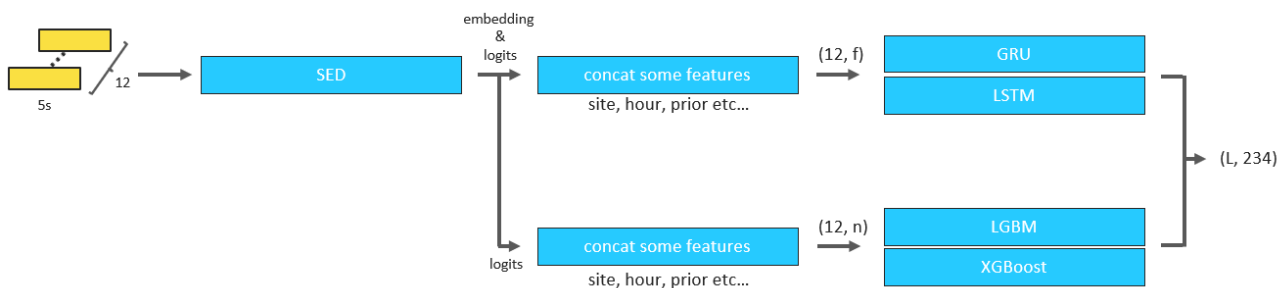
For non-sonotype labels:

| Branch   | Type | Mode     | Weight |
|----------|------|----------|--------|
| GRU      | NN   | residual | 0.5    |
| LSTM     | NN   | residual | 1.0    |
| LightGBM | tree | residual | 0.5    |
| XGBoost  | tree | residual | 1.0    |

For sonotype labels:

| Branch | Type | Mode   | Weight |
|--------|------|--------|--------|
| GRU    | NN   | direct | 0.5    |
| LSTM   | NN   | direct | 0.5    |

## Inference



Inference ran on 60-second soundscape files and produced 12 submission rows per file.

I exported the SED backbones to OpenVINO IR and ran them on CPU. The SED heads and NN stackers stayed in PyTorch.

The inference flow was:

1. Split each 60-second file into 5-second chunks.
2. Build mel spectrograms.
3. Run OpenVINO SED backbones.
4. Run PyTorch SED heads.
5. Aggregate segmentwise predictions to 5-second submission rows.
6. Build stacker features from base logits, embeddings, priors, metadata, and temporal context.
7. Apply the stacker ensemble.
8. Apply lightweight post-processing.

## Post-processing

I used two post-processing steps.

First, I applied temporal smoothing with weights `[0.1, 0.2, 0.4, 0.2, 0.1]`: for each class, each 5-second row was replaced by a weighted average of nearby rows centered on the current row.

Second, for sonotype labels, I grouped related sonotype variants and set every label in each group to the maximum score within that group.

## Stepwise LB Gains

I also measured the contribution of the main components. The deltas below are relative to the previous row.

| Stage                | Public LB | Δ Public | Private LB | Δ Private |
|----------------------|-----------|----------|------------|-----------|
| Baseline             | 0.90870   | -        | 0.92496    | -         |
| + Perch distillation | 0.91565   | +0.00695 | 0.94070    | +0.01574  |
| + Pseudo-labeling    | 0.93608   | +0.02043 | 0.94875    | +0.00805  |



| Stage      | Public LB | $\Delta$ Public | Private LB | $\Delta$ Private |
|------------|-----------|-----------------|------------|------------------|
| + Stackers | 0.94751   | +0.01143        | 0.95675    | +0.00800         |

## What Didn't Work for Me

---

- Fine-tuning SED models on data from previous BirdCLEF competitions.
- Multiple rounds of pseudo-labeling for the SED models.
- Using labels predicted by Perch during distillation.

## Closing

---

For me, this competition was not solved by simply making a stronger 5-second acoustic model. The base SEDs were important, but the second-stage modeling of soundscape context, site/hour priors, temporal continuity, and sonotype-specific behavior was just as important.

Thanks again to the hosts and to everyone who shared discussions, notebooks, and experiments.

As a side note, almost all of the implementation was done with Codex. I still had to design the experimental plan myself, but I could not have won gold without it. I also wrote this solution write-up together with Codex. Thank you, Codex!

[inference\\_code](#)